

Sep 07, 22 18:53	main.py	Page 1/4
<pre>#!/bin/env python3.8 """ Michael Bentivegna Deep Learning Assignment 1 Professor Curro """ import os import logging import matplotlib import matplotlib.pyplot as plt import numpy as np import tensorflow as tf from absl import app from absl import flags from tqdm import trange from dataclasses import dataclass, field, InitVar script_path = os.path.dirname(os.path.realpath(__file__)) @dataclass class LinearModel: # Each of the necessary variables for the model weights: np.ndarray gaussian_means: np.ndarray gaussian_sd: np.ndarray bias: float @dataclass class Data: rng: InitVar[np.random.Generator] num_features: int num_samples: int sigma: float x: np.ndarray = field(init=False) y: np.ndarray = field(init=False) def __post_init__(self, rng): """ Get Noisy Samples of y = sin(2*pi*x) """ self.index = np.arange(self.num_samples) self.x = rng.uniform(0, 1, size=(self.num_samples, self.num_features)) clean_y = np.sin(2 * np.pi * self.x) self.y = rng.normal(loc=clean_y, scale=self.sigma) def get_batch(self, rng, batch_size): """ Select random subset of examples for training batch """ choices = rng.choice(self.index, size=batch_size) return self.x[choices], self.y[choices].flatten() font = {"size": 10} matplotlib.style.use("classic")</pre>		

Sep 07, 22 18:53	main.py	Page 2/4
<pre>matplotlib.rc("font", **font) # Define default values for flags FLAGS = flags.FLAGS flags.DEFINE_integer("num_features", 1, "Number of features in record") flags.DEFINE_integer("num_samples", 50, "Number of samples in dataset") flags.DEFINE_integer("num_gaussians", 5, "Number of gaussians (M value)") flags.DEFINE_integer("batch_size", 16, "Number of samples in batch") flags.DEFINE_integer("num_iters", 300, "Number of SGD iterations") flags.DEFINE_float("learning_rate", 0.1, "Learning rate / step size for SGD") flags.DEFINE_integer("random_seed", 31415, "Random seed") flags.DEFINE_float("sigma_noise", 0.1, "Standard deviation of noise random variable") flags.DEFINE_bool("debug", False, "Set logging level to debug") def gaussianModel(x, mu, sigma): """ Get output value of gaussian function """ return tf.math.exp(-((x - mu) ** 2) / sigma**2) class Model(tf.Module): def __init__(self, num_gaussians, rng,): """ A linear regression model with gaussian basis functions """ self.num_gaussians = num_gaussians self.w = tf.Variable(rng.normal(shape=[1, num_gaussians])) self.mean = tf.Variable(tf.linspace(0.0, 1.0, num=num_gaussians)) self.sd = tf.Variable([0.1] * num_gaussians) self.b = tf.Variable(0.0) def __call__(self, x): """ Get the y_hat value for each sample """ total = tf.zeros(shape=(len(x), self.num_gaussians)) total += self.w * gaussianModel(x, self.mean, self.sd) return tf.reduce_sum(total, 1) + self.b @property def model(self): """ Check values after training """ return self.w.numpy(), self.mean.numpy(), self.sd.numpy(), self.b.numpy() def main(a): logging.basicConfig() if FLAGS.debug: logging.getLogger().setLevel(logging.DEBUG) # Safe np and tf PRNG</pre>		

Sep 07, 22 18:53

main.py

Page 3/4

```

seed_sequence = np.random.SeedSequence(FLAGS.random_seed)
np_seed, tf_seed = seed_sequence.spawn(2)
np_rng = np.random.default_rng(np_seed)
tf_rng = tf.random.Generator.from_seed(tf_seed.entropy)

# Get sample data
data = Data(
    np_rng,
    FLAGS.num_features,
    FLAGS.num_samples,
    FLAGS.sigma_noise,
)

# Initialize model
model = Model(FLAGS.num_gaussians, tf_rng)
optimizer = tf.optimizers.SGD(learning_rate=FLAGS.learning_rate)

# Use stochastic gradient descent for training during each iteration
bar = trange(FLAGS.num_iters)
for i in bar:
    with tf.GradientTape() as tape:
        x, y = data.get_batch(np_rng, FLAGS.batch_size)
        y_hat = model(x)
        loss = 0.5 * tf.reduce_mean((y_hat - y) ** 2)

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(
        [
            (grad, var)
            for (grad, var) in zip(grads, model.trainable_variables)
            if grad is not None
        ]
    )

    bar.set_description(f"Loss @ {i} => {loss.numpy():0.6f}")
    bar.refresh()

if FLAGS.num_features > 1:
    # Only continue to plotting if x is a scalar
    exit(0)

logging.debug(model.model)

# Plotting regression model and basis functions
fig, ax = plt.subplots(1, 2, figsize=(10, 3), dpi=200)

ax[0].set_title("Fit")
ax[0].set_xlabel("x")
ax[0].set_ylim(np.amin(data.y) * 1.5, np.amax(data.y) * 1.5)
h = ax[0].set_ylabel("y", labelpad=10)
h.set_rotation(0)
xs = np.linspace(0, 1, 1000)
xs = xs[:, np.newaxis]
ax[0].plot(
    xs,
    np.squeeze(model(xs)),
    "r--",
    xs,
    np.sin(2 * np.pi * xs),
    "b-",
    np.squeeze(data.x),
    data.y,

```

Sep 07, 22 18:53

main.py

Page 4/4

```

    "go",
)

ax[1].set_title("Basis Functions")
ax[1].set_xlabel("x")
h = ax[1].set_ylabel("y", labelpad=10)
h.set_rotation(0)
ax[1].set_ylim([0, 1])
for j in range(FLAGS.num_gaussians):
    ax[1].plot(xs, gaussianModel(xs, model.mean[j], model.sd[j]))
plt.tight_layout()
plt.savefig(f"{script_path}/fit.pdf")

if __name__ == "__main__":
    app.run(main)

```